



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DHL DAC 3.0 REPORT

AI-Powered Knowledge Base Automation for DHL Logistics Operations

Prepared by	Tan Zheng Yu
Matric Number	A23CS5017
Course	SECJ3483 Web Technology
Lecturer	En. Norizam Bin Katmon
Submission Date	14/5/2026

1. Introduction and Scenario Description	5
1.1 Logistics Context	5
1.2 Selected Scenario	5
1.3 Problem Statement	5
1.4 Project Objective	6
2. System Architecture and Design	7
2.1 Architectural Overview	7
2.2 Major Components	8
2.3 Design Logic and Business Logic Rationale	8
2.4 End-to-End Manual Workflow	9
2.5 Roles and Governance	10
3. Web Application and API Implementation Analysis	10
3.1 A1. System Design and Business Logic	10
3.2 A2. User Interface and Interaction	11
3.3 A3. JavaScript Functionality	12
3.4 A4. Data Management and Workflow	13
3.5 A5. JSON and API Integration	14
4. Technical Explanation: API, Data, and JSON	15
4.1 API Behavior	15
Manual upload pattern	15
RPA ingest pattern	15
4.2 Core JSON Structure	16
4.3 Upload Response Example	16
4.4 Processing Status Response Example	17
4.5 RPA Ingest Response Example	17
4.6 Data Handling Pipeline	18
4.7 Long Document Preservation	19
4.8 Versioning and Auditability	19
4.9 Automated Verification in the Repository	19
5. RPA Workflow and Integration Explanation	19
5.1 B1. RPA Process Identification and Justification	19
5.2 B2. RPA Workflow Design	20
5.3 B3. Automation Logic and Trigger Design	22
Trigger logic	22
Step-by-step automation logic	22
5.4 B4. RPA Integration and Explanation	22
Integration contract	22
Integration behavior	23
Status transition semantics	23
UiPath component understanding	23
6. UI Screenshots and Explanation	24
Screenshot 1. Login Page	24
Screenshot 2. Dashboard Overview	24
Screenshot 3. Upload Console and Processing Tracker	25

Screenshot 4. AI Draft Builder	26
Screenshot 5. Review Queue	26
Screenshot 6. Knowledge Base Library	27
Screenshot 7. Published Article Detail	27
Screenshot 8. Version History	27
7.1 Prerequisites	29
7.2 Backend Setup	29
7.3 Start the Backend	29
7.4 Start the Frontend	30
7.5 Optional Combined Startup	30
7.6 Demo Accounts	30
7.7 Basic Web Application Usage	30
Editor workflow	30
Reviewer workflow	31
Reader workflow	31
Audit workflow	31
7.8 UiPath Setup and Execution	31
7.9 Testing	32
7.10 Operational Notes	33
8. Conclusion	33

Abstract

The DHL Knowledge Base Automation project addresses a realistic logistics knowledge-management problem: operational knowledge is frequently trapped in unstructured text files, emails, screenshots, and ad hoc notes rather than preserved as governed, reusable knowledge assets. The repository implements a web-based knowledge workflow and a supporting RPA ingestion flow. The web system provides authentication, source upload, text extraction, OCR-assisted processing, AI-assisted draft generation, article editing, reviewer control, publication, search, and version history. The RPA layer, implemented in UiPath, performs controlled file ingestion, backend submission, result parsing, file routing, and run summarization through a final message box. Together, these components demonstrate a practical human-in-the-loop architecture suitable for DHL operations, where automation accelerates ingestion while editorial and reviewer controls preserve knowledge quality and governance.

1. Introduction and Scenario Description

1.1 Logistics Context

DHL logistics operations depend on fast access to procedural knowledge. In practice, however, operational knowledge is often distributed across many inconsistent sources: short troubleshooting notes, exported email threads, customer onboarding instructions, screenshot-based advice, and long SOP drafts. This creates a recurring operational problem. Valuable knowledge exists, but it is difficult to standardize, search, audit, and reuse. When a problem recurs, staff members may need to rediscover a solution from old messages instead of retrieving an approved article from a knowledge repository.

The sample data in docs/data_source/ demonstrates this context clearly. It contains access-error notes, label-printing problems, POD upload instructions, invoice-mismatch emails, onboarding checklists, and screenshot-based routing guidance. These are precisely the types of fragmented, semi-structured operational inputs common in a logistics environment.

1.2 Selected Scenario

The selected scenario is AI-assisted knowledge capture for DHL logistics operations. The system accepts messy operational source material and transforms it into structured draft SOPs or knowledge-base articles that can be reviewed, versioned, and published through a controlled web application.

Representative repository scenarios include:

- resolving AUTH_401 module access errors
- correcting customer address validation problems
- fixing POD upload failures caused by large image files
- standardizing new staff system setup steps
- preserving long onboarding procedures for credit approval and customer setup

1.3 Problem Statement

The business problem is not simply document storage; it is knowledge governance. In a logistics operation, the absence of structured knowledge causes:

- repeated operational errors
- slow issue resolution
- inconsistent staff onboarding
- dependency on experienced staff rather than documented procedures
- weak traceability of content changes and approvals

Therefore, the solution must do more than collect files. It must extract content, normalize it, structure it into reusable knowledge, preserve provenance, and keep human oversight in the final approval stages.

1.4 Project Objective

The objective of this project is to build an end-to-end knowledge automation pipeline in which:

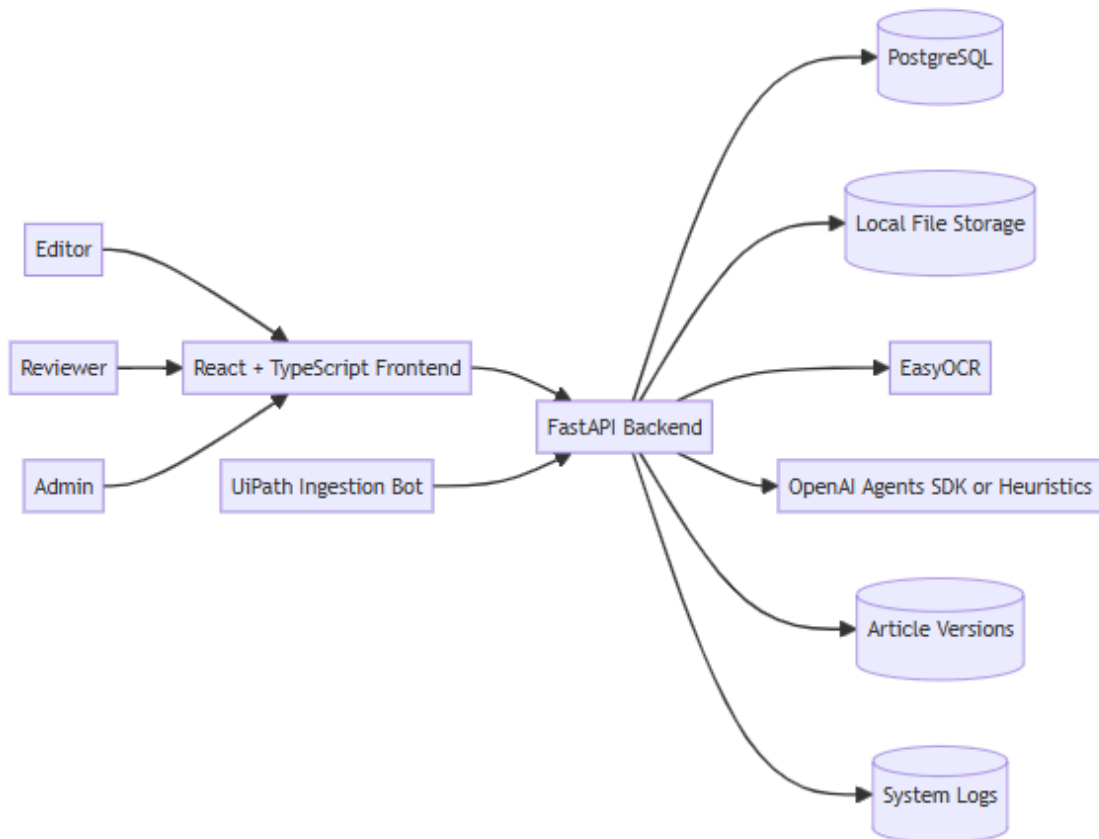
1. users or RPA submit operational source files
2. the backend extracts text and checks duplicates
3. OCR is applied to image-based inputs when needed
4. AI or local heuristics structure the content into a validated JSON article draft
5. editors review and refine the draft
6. reviewers approve, reject, request changes, or publish the result
7. the final article becomes part of a searchable internal knowledge base

This objective aligns strongly with the logistics scenario because it combines automation for repetitive ingestion tasks with human review for judgment-intensive publication decisions.

2. System Architecture and Design

2.1 Architectural Overview

The repository implements a layered architecture in which the frontend, backend, database, OCR service, AI service, and RPA workflow each have a clear responsibility.



This architecture is appropriate for the chosen scenario because it separates concerns cleanly:

- the frontend manages interaction and role-based access
- the backend owns business logic and workflow state
- the database persists articles, sources, versions, OCR records, and logs
- the RPA layer performs operational ingestion rather than business decision-making

2.2 Major Components

Component	Technology	Architectural role
Web frontend	React 19, TypeScript, Vite, Tailwind CSS, shadcn/ui	authentication UI, upload workflow, draft editing, review queue, knowledge browsing, version viewing
Backend API	FastAPI, SQLAlchemy, Pydantic	upload ingestion, source processing orchestration, article CRUD, workflow transitions, role enforcement
Database	PostgreSQL schema via SQLAlchemy models	persistence for users, source documents, OCR results, article records, keywords, versions, and logs
File storage	local filesystem	retained storage for uploaded source files
OCR layer	EasyOCR	text extraction from screenshots and images
AI layer	OpenAI Agents SDK or local heuristics	structured article generation into <code>KBArticleDraft</code>
RPA layer	UiPath Studio	folder-based ingestion, backend submission, response handling, and file routing

2.3 Design Logic and Business Logic Rationale

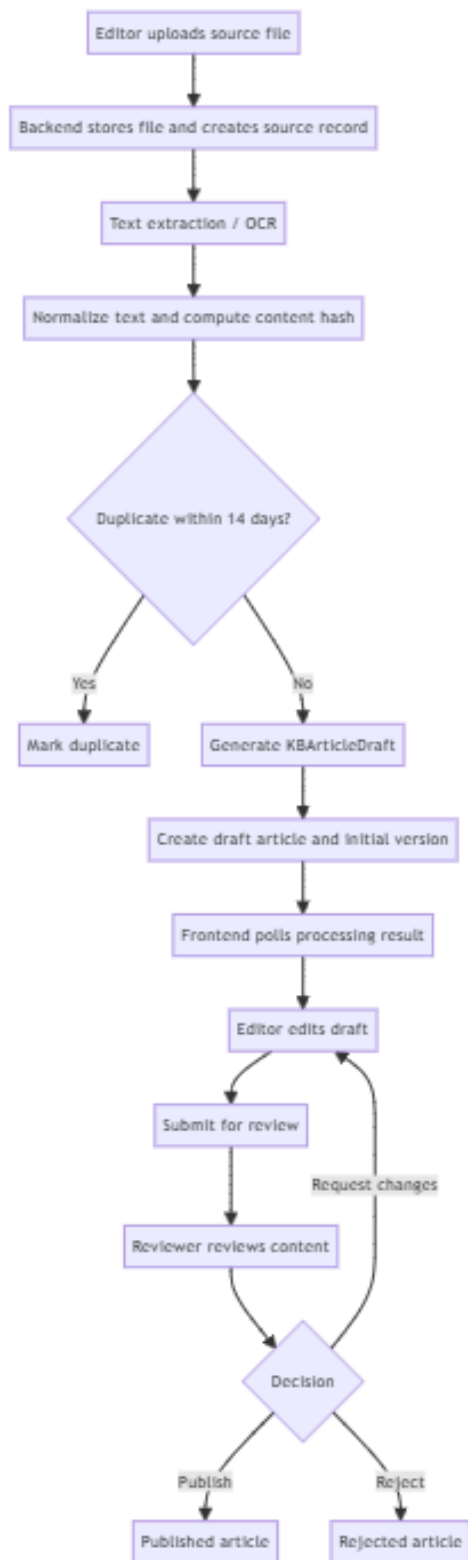
The system demonstrates strong alignment between technical design and business logic.

First, the upload flow is asynchronous for manual users. The endpoint `POST /api/uploads` returns `202 Accepted` with a `processing_id`, and the frontend polls `GET /api/processing/{processing_id}` until a terminal state is reached. This is an appropriate design choice because extraction, OCR, and AI generation are not guaranteed to finish within a normal synchronous request window.

Second, the RPA ingestion flow is synchronous by design. The endpoint `POST /api/rpa/ingest` waits for processing to finish and returns a contract-shaped final result such as `created`, `duplicate`, `needs_editor_review`, or `failed`. This is appropriate for UiPath because the robot must make an immediate routing decision after each submission.

Third, the workflow preserves governance. Automation accelerates source intake, but content publication remains controlled by article status transitions. A knowledge item moves through business states such as `draft`, `submitted`, `reviewed`, and `published`, which reflects the real need for review in logistics operations.

2.4 End-to-End Manual Workflow



2.5 Roles and Governance

Role	Responsibilities in the implemented system
Editor	upload sources, edit draft articles, save revisions, submit drafts for review
Reviewer	inspect review-stage articles, edit during review, publish, reject, or request changes
Admin	full access including deletion and direct editing of published content

The role model reinforces business governance. Editors prepare knowledge. Reviewers authorize it. Admins maintain the system boundary and content lifecycle. This is an appropriate control model for internal logistics knowledge.

3. Web Application and API Implementation Analysis

3.1 A1. System Design and Business Logic

The web application demonstrates a coherent operational workflow rather than a collection of isolated pages. The primary business journey is:

1. authenticate with a role-based account
2. upload a source file
3. monitor processing status
4. inspect the generated draft
5. edit title, summary, description, steps, sections, and keywords
6. submit the article for review
7. review, publish, reject, or return the article for changes
8. read the final article in the knowledge base
9. inspect version history for governance and traceability

This design is particularly strong because it mirrors a real knowledge-governance pipeline. The repository does not treat AI output as automatically publishable truth. Instead, the system uses `requires_editor_review` and status transitions to ensure that uncertain OCR results, ambiguous content, or long preserved documents are still subject to human control.

The backend business logic reinforces this design:

- source types are inferred from file extension and MIME type
- extracted text is normalized before hashing
- duplicates are detected using a 14-day lookback on normalized content hash
- OCR-based items can be flagged for review according to confidence thresholds
- long documents bypass aggressive AI restructuring and use preservation heuristics instead
- each article mutation creates a new version record

These decisions are well justified for the DHL scenario because they balance operational efficiency with auditability and content reliability.

3.2 A2. User Interface and Interaction

The frontend is organized as a role-aware workspace rather than a static multipage mock-up. Navigation includes:

- Dashboard
- Upload Console
- AI Draft Builder
- Review Queue
- Knowledge Base
- Version History
- RPA Runs
- Admin Settings

The user interaction model is strong in several ways.

First, the login screen is straightforward and role-aware. Demo credentials are presented clearly, reducing friction during evaluation or demonstration.

Second, the upload console provides immediate interaction feedback. Users select a supported file, submit it, receive a processing identifier, and observe live status updates in the processing tracker. This is more than a static upload form; it is a feedback-driven processing interface.

Third, the draft editor presents structured fields rather than an unbounded text area. This supports controlled editing of title, summary, description, steps, sections, keywords, and change notes, which is appropriate for SOP and KB authoring.

Fourth, the review queue separates editorial work from governance work. The reviewer sees readable article content and can request changes, reject, or publish the record. This enforces role separation in the interface itself.

For precision, the backend also exposes a dedicated `approve` endpoint and supports a `reviewed` intermediate state, while the current review screen operationalizes review mainly through edit, publish, request changes, and reject actions.

Fifth, the knowledge base view supports reading published content in a document-style layout, including source references, tags, and article metadata.

The UI also exhibits responsive state changes:

- upload progress is updated through polling
- search uses deferred query handling
- article lists reload after save, transition, and delete actions
- error messages are surfaced to the user instead of silently failing

An important accuracy note is that the core operational pages are implemented and dynamic, while RPA Runs and Admin Settings currently behave as placeholder shells. Similarly, several dashboard KPI widgets are illustrative cards rather than live analytics queries. This does not undermine the core workflow, but it distinguishes implemented operational pages from presentation-only modules.

3.3 A3. JavaScript Functionality

The frontend shows meaningful JavaScript and TypeScript logic rather than simple click handlers only.

Key evidence of strong application logic includes:

- `useEffect` is used for session restoration, article loading, upload polling, and global search
- `async/await` is used across upload, save, transition, delete, and fetch workflows
- `useDeferredValue` is used to smooth search-driven updates
- utility functions such as `textToSteps`, `textToSections`, and `splitCsv` transform user input into structured JSON payloads
- `Promise.all` is used to fetch article details and version history together
- status-based routing in the UI sends articles to the correct workspace such as Drafts, Review Queue, or Knowledge Base

From a rubric perspective, this is important because it demonstrates:

- event handling through form submissions and button actions
- asynchronous behavior through polling and API operations
- state synchronization between multiple views
- structured data transformation before persistence

The frontend is therefore not merely a view layer. It performs real client-side orchestration for structured editing and multi-step knowledge workflows.

3.4 A4. Data Management and Workflow

The system uses proper persistent data structures and supports a meaningful workflow rather than isolated CRUD rows.

The main persistent entities are:

- app_users
- source_documents
- attachments
- ocr_results
- kb_articles
- article_versions
- article_sources
- keywords
- article_keywords
- ai_generation_runs
- system_logs

This schema supports a rich workflow model:

- a source document represents the uploaded or RPA-submitted artifact
- an attachment preserves the stored file reference
- OCR results preserve extraction evidence for image-based sources
- a knowledge article stores the current approved or in-progress record
- article versions preserve history and change notes
- keywords are normalized into reusable tags
- AI generation runs and system logs preserve accountability and audit data

CRUD support is also clear:

- **POST** creates uploads and status transitions
- **GET** retrieves profiles, processing state, articles, article details, and versions
- **PATCH** updates structured article content
- **DELETE** removes articles for admin users

The system therefore exceeds simple CRUD by embedding a controlled workflow on top of persistent article records.

3.5 A5. JSON and API Integration

The repository demonstrates strong API integration. Core article content is not hardcoded in the operational workflow. Instead, it is created, retrieved, updated, and deleted through JSON APIs backed by database persistence.

Implemented endpoint coverage includes:

Method	Endpoint	Purpose
POST	/api/auth/login	login and token issuance
GET	/api/users/me	current user profile
POST	/api/uploads	manual source upload
GET	/api/processing/{processing_id}	asynchronous processing status
POST	/api/rpa/ingest	synchronous RPA ingestion
GET	/api/articles	article listing and search
GET	/api/articles/{article_id}	article detail
PATCH	/api/articles/{article_id}	structured article update
DELETE	/api/articles/{article_id}	admin deletion

POST	/api/articles/{article_id}/submit-review	draft submission
POST	/api/articles/{article_id}/approve	reviewer approval
POST	/api/articles/{article_id}/publish	publish article
POST	/api/articles/{article_id}/reject	reject article
POST	/api/articles/{article_id}/request-changes	return article to draft
GET	/api/articles/{article_id}/versions	version history

The frontend consumes these endpoints through a typed Axios client in `frontend/src/lib/api.ts`. This is a strong implementation pattern because it centralizes request logic, response typing, and authorization handling.

For accuracy, the core article workflow is API-driven and non-hardcoded, whereas some dashboard metrics remain illustrative constants. This means the operational CRUD features satisfy the JSON/API integration objective even though not every dashboard card is backed by a live analytics endpoint.

4. Technical Explanation: API, Data, and JSON

4.1 API Behavior

The system deliberately uses two different ingestion patterns.

Manual upload pattern

POST `/api/uploads` is asynchronous.

1. the backend stores the file and creates a source record
2. the API returns `processing_id`, `processing_status`, and `processing_stage`
3. the frontend polls `/api/processing/{processing_id}`
4. the final response includes source metadata, optional OCR information, draft preview data, and the generated article

This pattern is appropriate because OCR and AI generation are potentially slow.

RPA ingest pattern

POST `/api/rpa/ingest` is synchronous.

1. UiPath submits the file
2. the backend executes the same extraction and generation pipeline
3. the endpoint waits for completion
4. the API returns a contract-shaped final result suitable for robot routing

This pattern is appropriate because RPA needs an immediate outcome for branch execution.

4.2 Core JSON Structure

The central data contract is `KBArticleDraft`, which formalizes the article generated by AI or heuristics.

```
{
  "schema_version": "1.0",
  "title": "Resolve AUTH_401 Access Failure",
  "kind": "article",
  "summary": "Validated draft for missing AD group access in forwarding operations.",
  "description": "Users cannot access the forwarding module because their access assignment is incomplete.",
  "steps": [
    { "step_no": 1, "instruction": "Check the user's AD group membership." },
    { "step_no": 2, "instruction": "Assign the missing forwarding access group." }
  ],
  "sections": [
    { "heading": "Cause", "content": "The required AD group access was not assigned." }
  ],
  "keywords": ["authorization", "auth-401", "forwarding"],
  "source_reference": "Error_Code_AUTH_401.txt",
  "source_type": "text",
  "requires_editor_review": false
}
```

This structure is technically strong for three reasons:

1. it is schema-validated through Pydantic
2. it is suitable for direct storage in JSON columns
3. it maps naturally to a web editing form

4.3 Upload Response Example



```
{
  "processing_id": "source-document-uuid",
  "processing_status": "pending",
  "processing_stage": "queued"
}
```

4.4 Processing Status Response Example

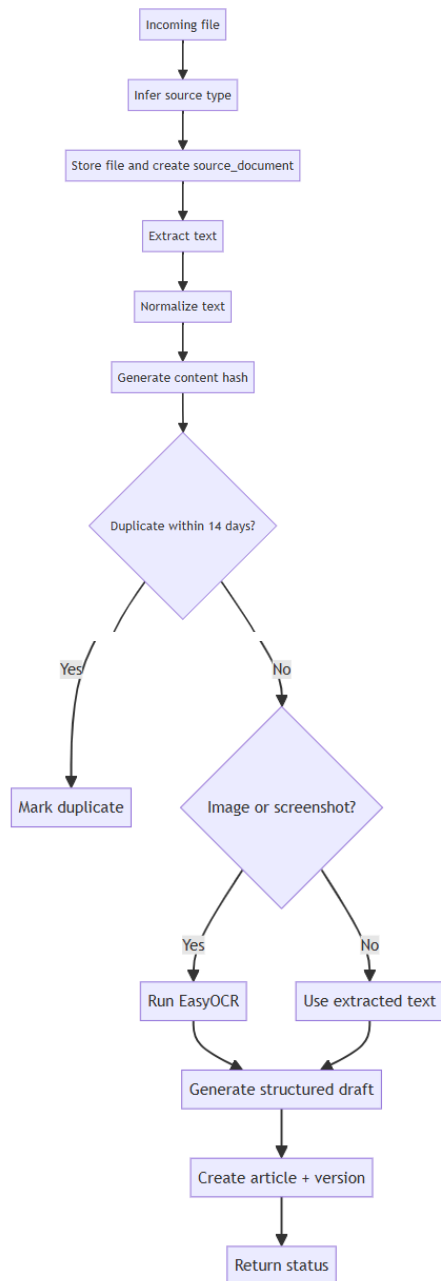
```
{
  "processing_id": "source-document-uuid",
  "processing_status": "created",
  "processing_stage": "completed",
  "source_document": {
    "id": "source-document-uuid",
    "original_filename": "Error_Code_AUTH_401.txt",
    "source_type": "text",
    "requires_editor_review": false
  },
  "generated_draft": {
    "title": "Resolve AUTH_401 Access Failure",
    "summary": "Steps to resolve missing access for a forwarding module.",
    "kind": "article",
    "source_reference": "Error_Code_AUTH_401.txt",
    "keywords": ["authorization", "auth-401"],
    "steps": [
      { "step_no": 1, "instruction": "Check the user's AD group membership." }
    ]
  },
  "article": {
    "id": "article-uuid",
    "kind": "article",
    "status": "draft",
    "structured_content": {}
  }
}
```

4.5 RPA Ingest Response Example

```
{
  "status": "created",
  "processing_id": "source-document-uuid",
  "article_id": "article-uuid",
  "source_document_id": "source-document-uuid",
  "duplicate_of_source_id": null,
  "message": "Draft article created",
  "requires_editor_review": false
}
```

4.6 Data Handling Pipeline

The backend processing pipeline is technically well defined.



The extraction layer supports:

- plain text decoding
- PDF parsing through `pypdf`
- DOCX parsing through `python-docx`
- email extraction from `.eml` and `.msg`
- EasyOCR extraction for image-based sources

4.7 Long Document Preservation

An especially strong design decision is the long-document preservation mode. If source text exceeds the configured `ai_long_document_char_limit`, the backend does not force a short-form AI rewrite. Instead, it preserves the original content through local heuristics and sets `requires_editor_review = true`. This is highly appropriate for long SOP material because it avoids losing procedural detail through over-compression.

4.8 Versioning and Auditability

Every meaningful article update creates a version snapshot. This includes:

- the initial generated draft
- editorial revisions
- workflow status transitions

As a result, the system supports governance, rollback analysis, and change accountability, all of which are important in operations documentation.

4.9 Automated Verification in the Repository

The repository includes backend test suites covering:

- database connectivity
- upload processing
- email extraction
- duplicate detection
- article generation and versioning
- synchronous RPA ingestion
- optional real OCR and real OpenAI integration cases

This strengthens the technical credibility of the project because the workflow is not described only in prose; it is also exercised through automated tests.

5. RPA Workflow and Integration Explanation

5.1 B1. RPA Process Identification and Justification

The chosen RPA process is well suited to automation because it is repetitive, file-based, deterministic, and operationally frequent. In DHL operations, new knowledge often appears as files placed into a monitored folder or exported mailbox location. The repetitive part is not the human judgment about

article quality; it is the mechanical work of:

- discovering incoming files
- checking whether they are eligible for ingestion
- submitting them to the system
- reading the returned status
- moving the files into the correct folder
- summarizing the run

This is an appropriate RPA target because the task is:

- rule-based
- high-frequency
- structured around standard file operations and HTTP calls
- low value if performed manually

By contrast, final article publication is not a good RPA target. It requires judgment, domain interpretation, and governance. The project therefore makes a sound architectural decision: UiPath automates ingestion, but the backend and human reviewers own knowledge quality and lifecycle decisions.

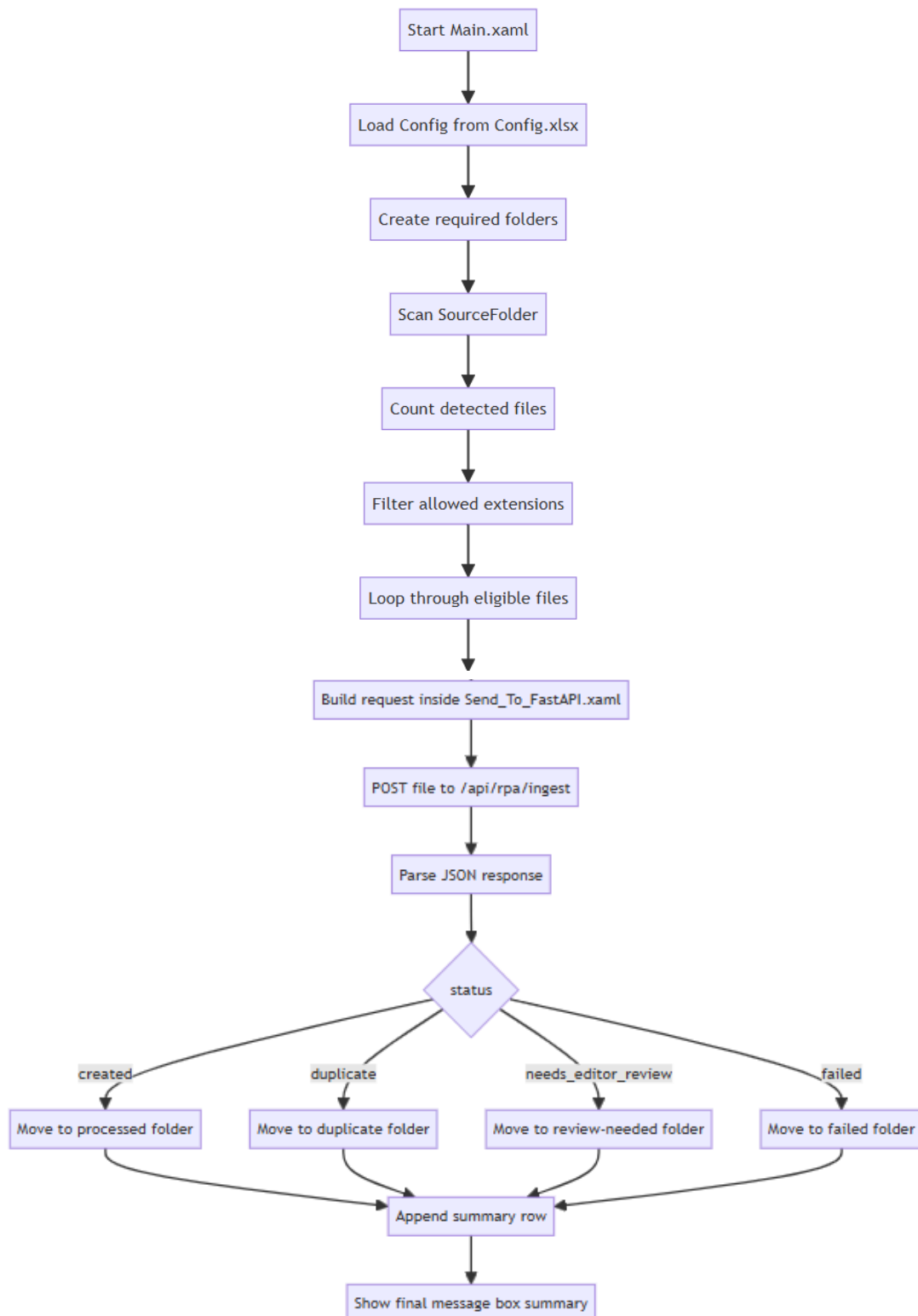
5.2 B2. RPA Workflow Design

The UiPath project is structured into reusable workflow files:

- Main.xaml
- Init_Config.xaml
- Scan_Source_Folder.xaml
- Filter_Eligible_Files.xaml
- Send_To_FastAPI.xaml
- Handle_Backend_Result.xaml
- Route_File.xaml

Although some earlier planning documents referred to a separate request-building workflow, the current repository embeds request construction directly inside `Send_To_FastAPI.xaml`. This is an implementation detail, but it is still a coherent design because the responsibility remains clearly separated.

The implemented RPA workflow is as follows:



This workflow is structured, branch-based, and operationally accurate. It captures the essential governance requirement that RPA should not publish articles or bypass the backend. Instead, it routes

source files based on backend authority.

5.3 B3. Automation Logic and Trigger Design

The implemented automation logic is stepwise and clear.

Trigger logic

The practical trigger is folder-based ingestion. When the UiPath project runs, it scans the configured SourceFolder and processes any files currently present. This is a realistic trigger for logistics operations because operational material is commonly accumulated in shared folders, exported mail locations, or synchronized drive directories.

Step-by-step automation logic

1. Init_Config.xaml reads configuration from Data/Config.xlsx.
2. The workflow initializes working folders such as source, processed, duplicate, failed, and review-needed locations.
3. Scan_Source_Folder.xaml retrieves all files from the configured source folder and stores the total count.
4. Filter_Eligible_Files.xaml narrows the set to approved file types.
5. Main.xaml iterates through the eligible files one by one.
6. Send_To_FastAPI.xaml constructs the request and posts the file to /api/rpa/ingest.
7. Handle_Backend_Result.xaml deserializes the returned JSON and extracts fields such as status, processing ID, article ID, and backend message.
8. Route_File.xaml increments the relevant counters, determines the correct destination folder, moves the file, and appends a summary row.
9. Main.xaml shows a final message box summarizing total detected files, eligible files, created count, duplicate count, needs review count, failed count, and per-file summary lines.

This automation logic covers ingestion, backend update, duplicate-aware routing, failure routing, and summary reporting, which are the most important operational automation requirements for the implemented scope.

5.4 B4. RPA Integration and Explanation

The RPA integration is technically strong because it uses the web application as the single source of truth instead of attempting direct database manipulation.

Integration contract

UiPath integrates through:

- POST /api/rpa/ingest

Returned data include:

- status
- processing_id
- article_id
- source_document_id
- duplicate_of_source_id
- message
- requires_editor_review

Integration behavior

The backend decides whether a file:

- creates a new article
- is a duplicate
- requires editor review
- fails processing

UiPath does not reinterpret the business rules. It simply consumes the contract and performs operational routing. This is excellent integration design because it preserves clean separation of concerns.

Status transition semantics

The integration also respects workflow semantics:

- a successful manual upload normally creates a draft
- a successful RPA ingest without review concerns can create an rpa_submitted article
- a processed but uncertain RPA item is preserved with requires_editor_review = true

This is important because it shows that RPA is not merely calling an API; it is participating in a governed lifecycle model.

UiPath component understanding

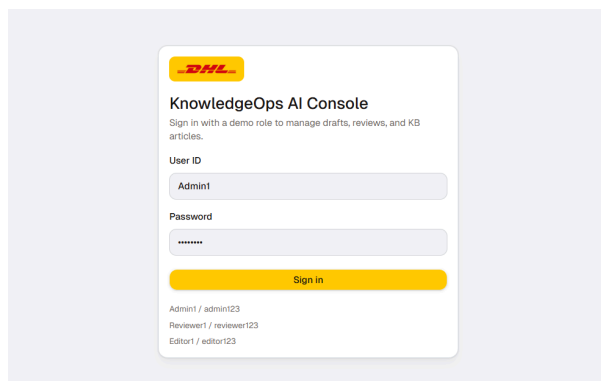
The workflow also demonstrates correct use of UiPath building blocks:

- Invoke Workflow File for modular orchestration
- Excel read activities for configuration loading
- HTTP Request activities for API integration
- JSON deserialization for response parsing
- file move activities for deterministic routing
- message box activities for end-of-run summarization

From a rubric perspective, this indicates a solid understanding of how UiPath should integrate with a JSON API-based web system.

6. UI Screenshots and Explanation

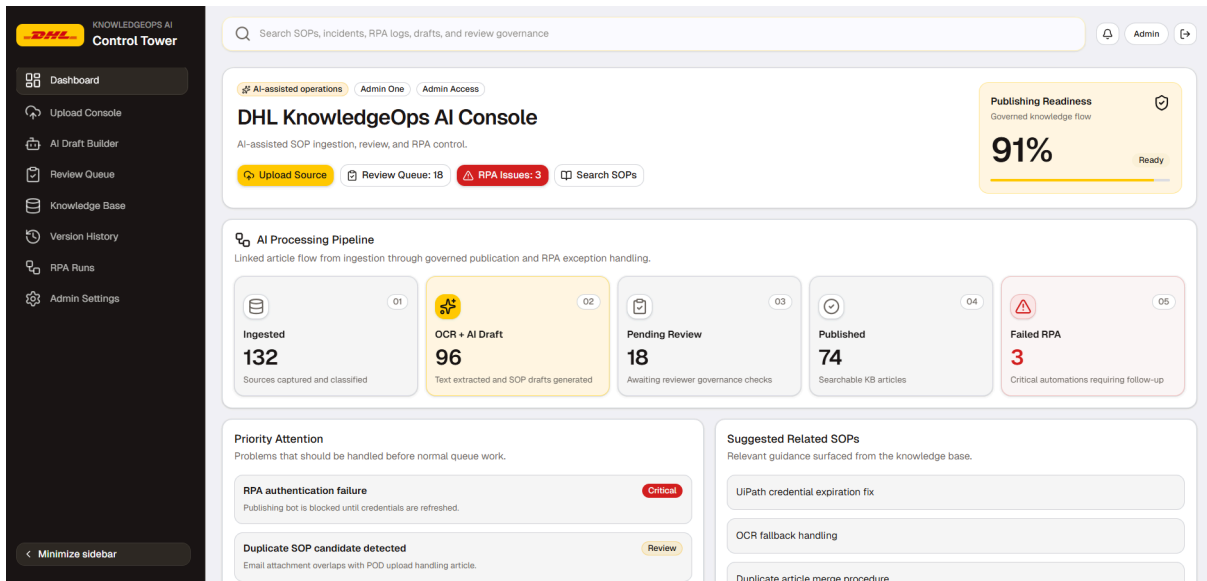
Screenshot 1. Login Page



Explanation:

This screenshot shows the DHL-branded login screen with the User ID field, Password field, sign-in button, and demo credentials. It demonstrates secure entry into the system and establishes the role-based nature of the interface.

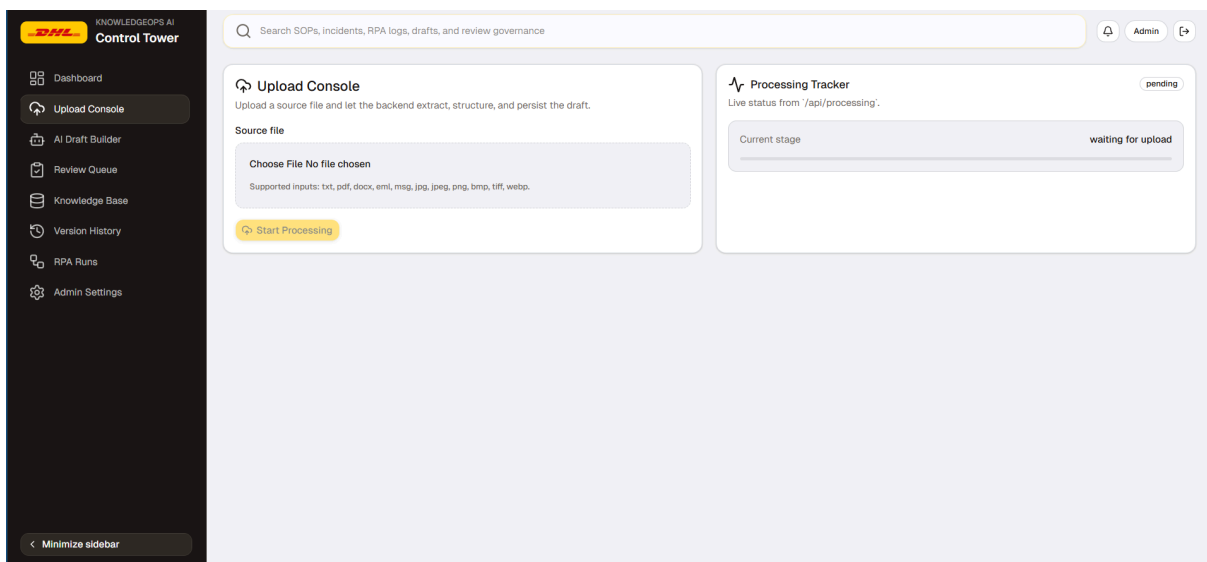
Screenshot 2. Dashboard Overview



Explanation:

This screenshot shows the dashboard, including the left navigation, role badge, overview widgets, and quick navigation buttons. It demonstrates overall workflow discoverability and shows how the system is presented as a unified operational console. It is acceptable to note in the caption that dashboard KPI cards are illustrative summary widgets rather than fully live analytics queries.

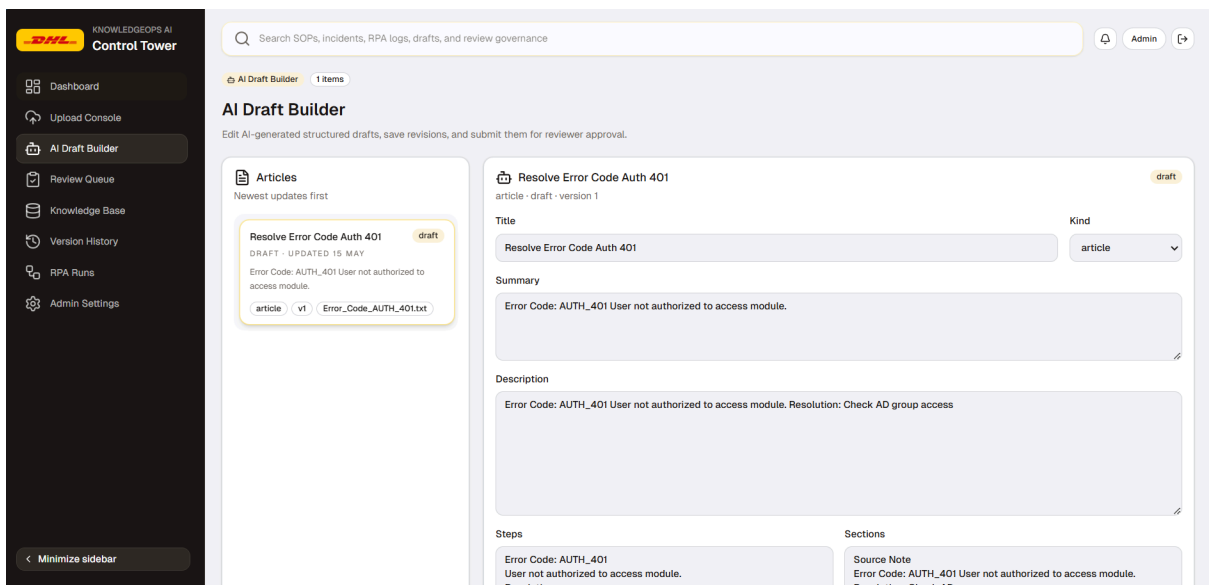
Screenshot 3. Upload Console and Processing Tracker



Explanation:

This is one of the most important screenshots because it demonstrates interactive file submission, supported source types, asynchronous status polling, and generated draft preview content. It provides strong evidence for UI interaction, asynchronous JavaScript behavior, and backend processing integration.

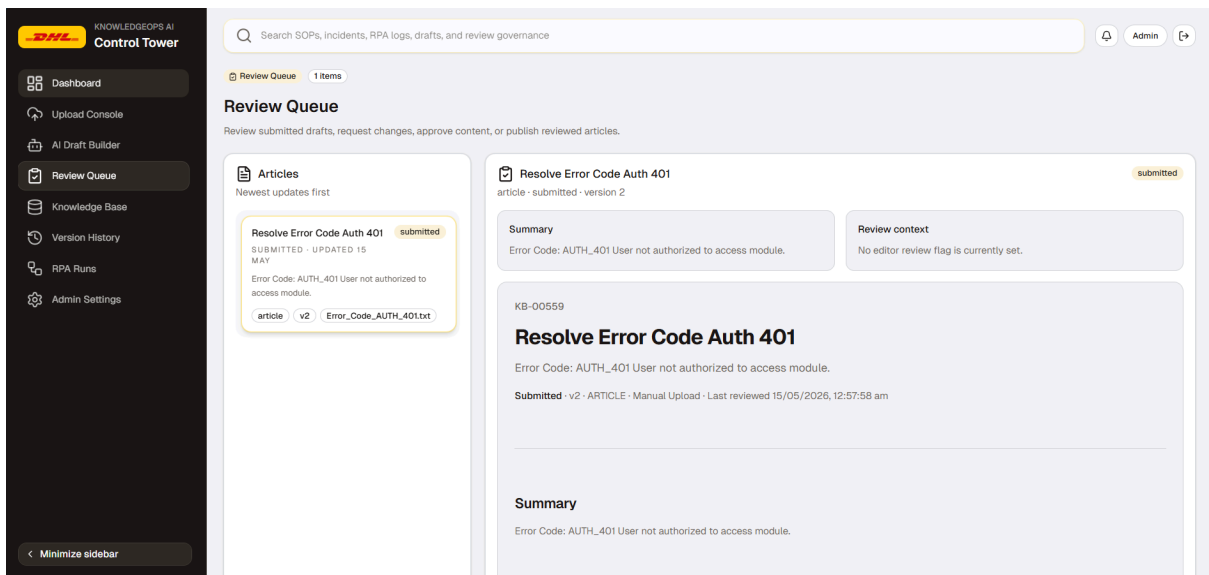
Screenshot 4. AI Draft Builder



Explanation:

This screenshot shows structured editing fields such as title, summary, description, steps, sections, keywords, and source reference. It demonstrates that the frontend is not a plain text viewer but a controlled editor for schema-based knowledge authoring.

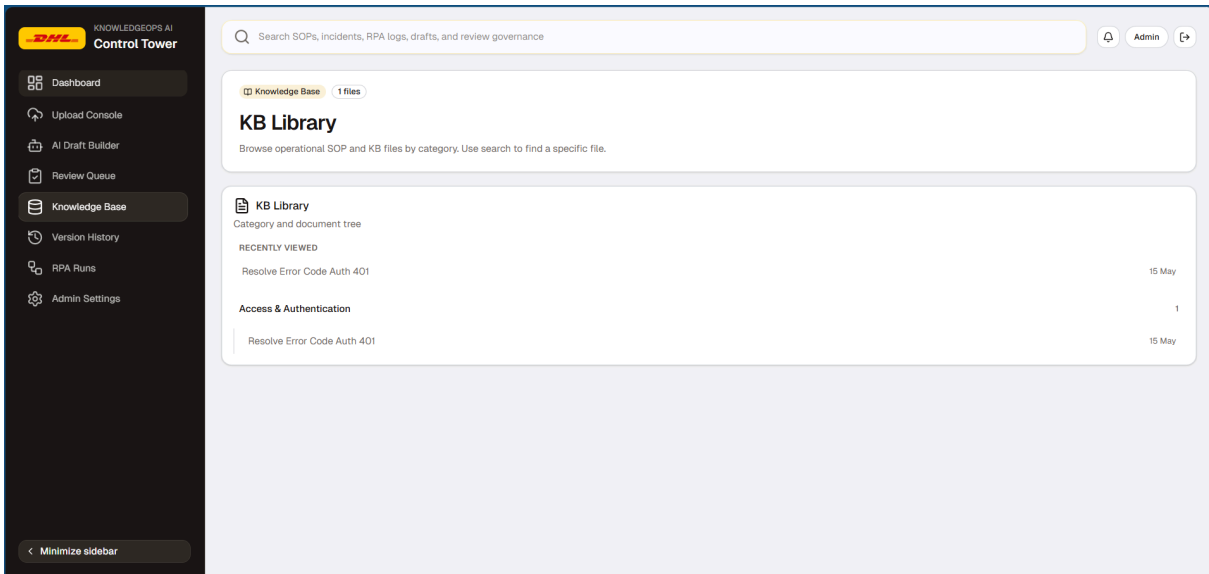
Screenshot 5. Review Queue



Explanation:

This screenshot shows the reviewer-facing article view together with actions such as edit, publish, request changes, and reject. It provides evidence of workflow governance and role separation between content preparation and publication.

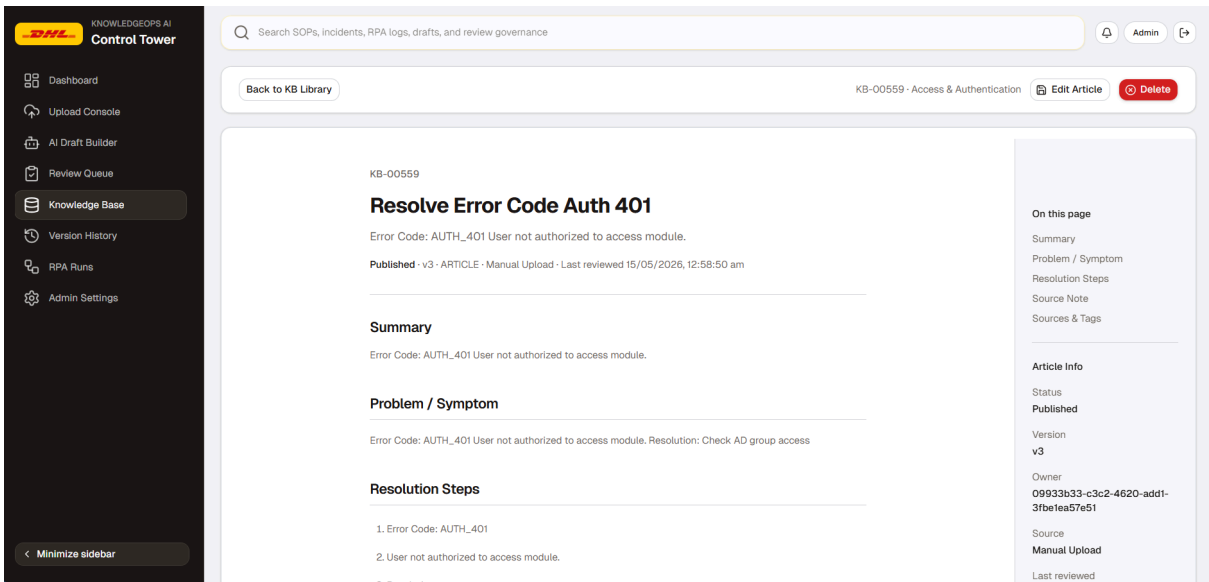
Screenshot 6. Knowledge Base Library



Explanation:

This screenshot shows the published article library or search results page. It demonstrates that the system supports knowledge retrieval rather than stopping at draft creation only.

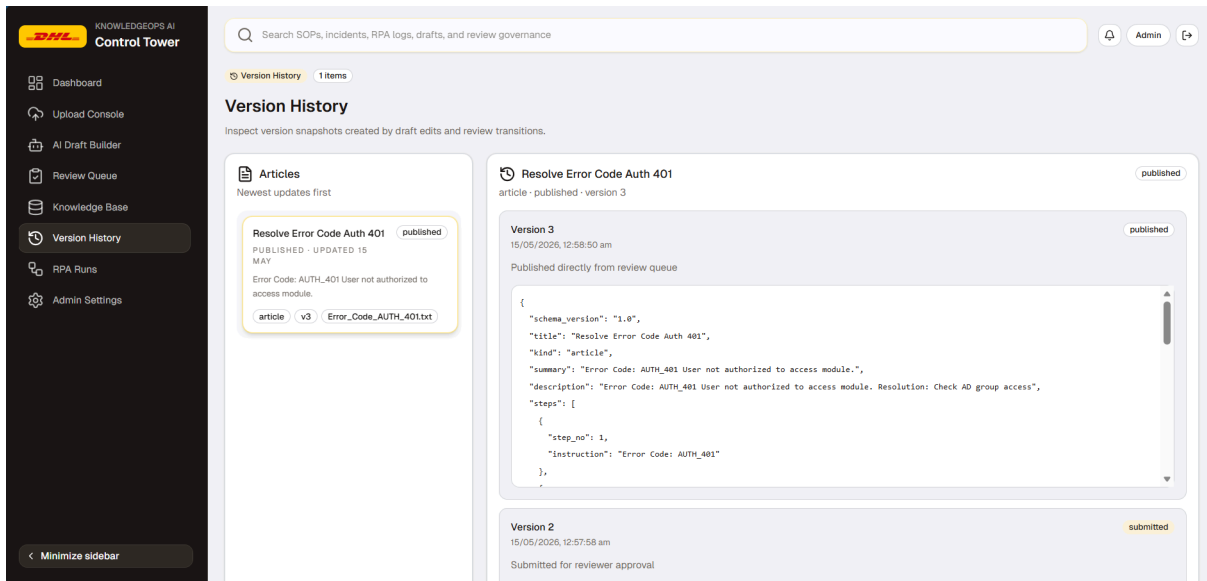
Screenshot 7. Published Article Detail



Explanation:

This screenshot shows a readable final knowledge article with summary, problem description, steps, sections, tags, and metadata. It demonstrates the final consumer-facing knowledge output.

Screenshot 8. Version History



Explanation:

This screenshot shows the saved version snapshots, status snapshots, timestamps, and change notes. It is important for demonstrating auditability and lifecycle governance.

7. User Manual

7.1 Prerequisites

Prepare the following before running the system:

- Node.js and npm
- Python 3.12+
- uv
- PostgreSQL

Optional but recommended:

- OPENAI_API_KEY for live OpenAI article generation
- EasyOCR dependencies for image and screenshot extraction

7.2 Backend Setup

From the backend folder:

```
Copy-Item .env.example .env
```

Update .env with the correct database values:

- DB_USER
- DB_PASSWORD
- DB_HOST
- DB_PORT
- DB_NAME

Optional AI/OCR related configuration may also be supplied through .env.

7.3 Start the Backend

```
cd backend  
uv run uvicorn app.main:app --reload
```

Default backend URL:

```
http://localhost:8000
```

7.4 Start the Frontend

```
cd frontend  
npm install  
npm run dev
```

Default frontend URL:

```
http://localhost:5173
```

7.5 Optional Combined Startup

From the repository root:

```
.\dev.ps1
```

7.6 Demo Accounts

The backend seeds local demo accounts automatically:

- Admin1 / admin123
- Reviewer1 / reviewer123
- Editor1 / editor123

7.7 Basic Web Application Usage

Editor workflow

1. sign in as Editor1

2. open Upload Console
3. upload a source file such as Error_Code_AUTH_401.txt
4. wait for the processing tracker to reach a terminal status
5. click Open Draft Builder
6. review and edit the generated article fields
7. save the draft
8. submit it for review

Reviewer workflow

1. sign in as Reviewer1
2. open Review Queue
3. select the submitted article
4. read the structured content
5. optionally add a review note
6. publish, reject, or request changes

Reader workflow

1. open Knowledge Base
2. search or browse for a published article
3. open the article detail page
4. review the final SOP or KB article

Audit workflow

1. open Version History
2. select the relevant article
3. review version numbers, timestamps, and change notes

7.8 UiPath Setup and Execution

The UiPath project is located at:

```
dhl-kb-automation-uipath\DHL.KB.Automation.Ingestion
```

Prepare the following before running the bot:

1. ensure the FastAPI backend is running
2. open the UiPath project in UiPath Studio
3. verify the configuration values in Data/Config.xlsx
4. ensure the configured source and destination folders exist
5. place test files in the configured SourceFolder
6. run Main.xaml

The UiPath workflow will:

1. load configuration
2. scan the source folder
3. filter eligible files
4. send each file to /api/rpa/ingest
5. parse the returned JSON
6. route the file into the correct folder
7. show the final summary in a message box

7.9 Testing

The backend repository includes automated tests.

Run from the backend folder:

```
uv run pytest
```

Useful variants:

```
uv run pytest -v -s -rs
uv run pytest tests/test_upload_processing.py
uv run pytest tests/test_rpa_ingest.py
```

Optional real integration tests:

- set RUN_REAL_OCR_TESTS=1 for OCR integration tests

- set `RUN_REAL_OPENAI_TESTS=1` and `OPENAI_API_KEY` for OpenAI integration tests

7.10 Operational Notes

The most important operational characteristics of the current implementation are:

- manual uploads use asynchronous polling
- RPA ingest returns a final synchronous result
- long documents may be preserved and flagged for review
- OCR-derived sources may require additional human review
- each save or workflow transition creates version history

8. Conclusion

The DHL Knowledge Base Automation repository demonstrates a convincing academic and practical solution to the logistics knowledge-capture problem. Its strongest qualities are the alignment between scenario and architecture, the use of structured JSON for article generation and persistence, the integration of AI and OCR into a governed workflow, and the separation of responsibilities between UiPath, FastAPI, and human reviewers.

From a rubric perspective, the project shows strong system logic, meaningful JavaScript behavior, proper API-backed CRUD and workflow control, and a well-justified RPA design. Most importantly, it does not automate for its own sake. It automates the repetitive ingestion layer while preserving human authority over knowledge quality and publication. That is precisely the kind of design discipline expected in a high-value logistics knowledge system.